

09-Network Communication(网络通信)

鸣谢：黑马程序员



💡 Tip

Previous Chapter(上一章): 08-Multithreading(多线程)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

Next Chapter(下一章): 10-Reflection(反射)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

• 09-Network Communication(网络通信)

一、概述

二、前置知识

1.基本通信架构

2.网络通信三要素

2.1 IP地址：设备在网络中的唯一标识

P1 IP地址的两种形式

P2 域名(Domain Name)

P3 公网和内网IP

P4 特殊IP

P5 常用的终端IP命令

2.2 端口号：应用程序在设备中的唯一标识

P1 端口分类

P2 注意事项

2.3 通信协议：设备连接和数据在网络中传输的规则

P1 UDP协议

P2 TCP协议

P1 通过三次握手建立可靠连接

P2 通过四次挥手断开连接

三、网络编程

1. `InetAddress` 类：代表IP地址

2.UDP通信

2.1 `DatagramSocket` 类：创建客户端和服务端

P1 构造器

P2 常用方法

2.2 `DatagramPacket`：创建数据包

P1 构造器

P2 常用方法

2.3 一发一收

2.4 多发多收

P1 IDEA中允许客户端多开的设置步骤

P2 代码实现

3A.单客户端TCP通信

3A.1 原理图

3A.2 `Socket` 类：开发客户端

P1 构造器

	P2 常用方法
3A.3	ServerSocket 类：开发服务端
	P1 构造器
	P2 常用方法
3A.4	代码演示（一发一收）
3A.5	单客户端多发多收注意事项
3B.	多客户端TCP通信
3B.1	3A代码的局限性
3B.2	重构思路：引入多线程
3B.3	代码实现
4.	TCP通信综合案例
4.1	即时通信群聊
	P1 原理图
	P2 代码实现
4.2	简易版BS架构
	P1 需求
	P2 原理图
	P3 代码实现
5.	拓展

一、概述

- 网络编程可以让设备中的程序和网络上其他设备中的程序进行网络通信，实现数据交互。
- `java.net` 包下提供了网络编程的解决方案。

二、前置知识

1.基本通信架构

- **CS架构**: Client-Server (客户端-服务器)
- **BS架构**: Browser-Server (浏览器-服务器)

2.网络通信三要素

2.1 IP地址：设备在网络中的唯一标识

P1 IP地址的两种形式

- **IPv4**: 32bits(4B), 一般采用点分十进制表示。如192.168.1.66。
- **IPv6**:
 - 128bits(16B), 一般采用冒分十六进制表示法, 即分成8段表示, 每段有16bits, 其中每4bits转成一个十六进制数表示, 段之间用冒号隔开。如
2001:0DB8:0000:0000:0008:0800:200C:417A。
 - 为了书写方便, IPv6还提供了压缩格式, 具体压缩规则为: 每组中的前导0都可以省略。因此上述地址可写为: 2001:0DB8:0:0:8:800:200C:417A
 - 如果存在连续两个及以上均为0的段, 可以用双冒号来代替, 因此上述地址可进一步简写为: 2001:0D88::8:800:200C:417A。
 - 需要注意的是, 在一个IPv6地址中只能使用一次双冒号, 否则当计算机将压缩地址恢复成完整地址时, 无法确定每个双冒号表示几个0。

P2 域名(Domain Name)

IP地址是数字形式, 可读性差且难以记忆, 因此人们发明了可读性较高且容易记忆的域名。

当用户想要访问某个网站时, 他在计算机设备上输入域名, 本地**DNS(Domain Name System)**服务器就会将域名解析成对应的IP地址并返回给设备, 此时设备就能通过网站的IP地址访问网站。(注: 此处没有考虑DNS缓存不命中的情况)

P3 公网和内网IP

- **公网IP**：可以连接互联网的IP地址。
- **内网IP**：也叫局域网IP，只能在组织或机构内部使用。192.168.开头的IP是常见的局域网IP。

P4 特殊IP

- 127.0.0.1，别名 `localhost`，代表本机IP，只会寻找当前所在的主机。

P5 常用的终端IP命令

- 查看本机IP地址：`ipconfig(Windows)/ifconfig(Linux)`。
 - 检查本机与目标IP之间的网络是否连通：`ping 目标IP`。
-

2.2 端口号：应用程序在设备中的唯一标识

16bits，表示范围是0~65535。

P1 端口分类

- **周知端口**：0~1023，被预先定义的知名应用所占用，比如HTTP占用80端口，FTP占用21端口。
- **注册端口**：1024~49151，分配给用户进程或某些应用程序，比如MySQL默认占用3306端口，Tomcat默认占用8080端口。
- **动态端口**：49152~65535，一般不固定分配给某种进程，而是采取动态分配的策略。

P2 注意事项

我们自己开发的程序一般使用注册端口，且同一设备中不能出现两个端口号相同的程序，否则会出错。

2.3 通信协议：设备连接和数据在网络中传输的规则

OSI网络参考模型	TCP/IP网络模型	各层对应	面向操作
应用层	应用层	HTTP、FTP、SMTP...	应用程序需要关注的：浏览器，邮箱。 程序员一般在这一层开发
表示层			
会话层			
传输层	传输层	UDP、TCP...	选择使用的TCP，UDP协议
网络层	网络层	IP...	封装源和目标IP
数据链路层	数据链路层+ 物理	比特流...	物理设备中传输
物理层			

P1 UDP协议

 Important

UDP(User Datagram Protocol): 用户数据报协议。

- **特点：**无连接、不可靠传输。
 - 不事先建立连接，数据以“包”为单位进行传输，“包”的数据部分一般包含：源IP、源程序端口号、目的IP、目的程序端口号、传输数据（64KB以内）等。
 - 发送方发送前不会确认接收方是否在线，数据在传输过程中丢失也不管，接收方收到数据后也不返回确认包ACK，因此传输不可靠。
- 但是UDP**通信效率高**，适用于语音通话、视频直播等对通信可靠性要求不高的场景。

P2 TCP协议

 Important

TCP(Transmission Control Protocol): 传输控制协议。

- **特点：**面向连接、可靠传输。
- 但是TCP通信效率相对不高。
- TCP采用三个步骤实现可靠传输：

1. 通过**三次握手**建立可靠连接；
2. 传输数据后进行确认，如果客户端没有收到服务器的确认或确认信息有误，则客户端必须重发；
3. 通过**四次挥手**断开连接。

P1 通过三次握手建立可靠连接

💡 Tip

可靠连接：通信双方收发信息正常无问题（全双工通信）。

SYN(Synchronize Sequence Numbers): 同步序列号，是TCP协议头部的一个控制标志位，用于发起连接请求并同步初始序列号。

- 当SYN=1且ACK=0时，表示纯连接请求；
- 当SYN=1且ACK=1时，表示对连接请求的确认。

第一次握手：客户端向服务器发送 `SYN=1, ACK=0` 的报文，携带随机生成的初始序列号 ISN，请求与服务器建立连接。

第二次握手：服务器收到客户端发来的连接请求报文后，回复 `SYN=1, ACK=1` 的报文，确认客户端的连接请求并发送自己的ISN。

第三次握手：客户端收到服务器发来的确认报文后，回复 `SYN=0, ACK=1` 的报文，确认服务器的ISN，连接建立完毕。

确认事项	第一次握手后，服务器得知	第二次握手后，客户端得知	第三次握手后，服务器得知
客户端发送能力正常	✓		
服务器接收能力正常		✓	
服务器发送能力正常		✓	
客户端接收能力正常			✓

P2 通过四次挥手断开连接

第一次挥手：主动关闭方向被动关闭方发送断开连接请求。

第二次挥手：被动关闭方返回一个响应：稍等。

第三次挥手：被动关闭方完成剩余数据传输后，返回一个响应：确认断开连接。

第四次挥手：主动关闭方向被动关闭方发送报文：收到剩余数据，正式确认断开连接。

三、网络编程

⚠ Warning

`try-with-resources` 是一把双刃剑：

- 优点是代码更简洁和健壮，同时防止资源泄露；
- 缺点是可能导致底层资源被意外关闭，不适用于需要长期保持连接的资源，如网络通信中的 `Socket` 流！

由于我编码时没有意识到这个缺点，故而以下代码均采用了这种方式自动关闭资源，请各位不要效仿！

1. `InetAddress` 类：代表IP地址

序号	方法	说明
01	<code>static InetAddress getLocalHost()</code>	将本机IP封装成一个 <code>InetAddress</code> 对象并返回。
02	<code>static InetAddress getByName(String host)</code>	根据IP或域名，返回一个 <code>InetAddress</code> 对象。
03	<code>String getHostName()</code>	获取该 <code>InetAddress</code> 对象的主机名（一般是域名）。
04	<code>String getHostAddress()</code>	获取该 <code>InetAddress</code> 对象的IP地址信息。
05	<code>boolean isReachable(int timeout)</code>	在指定时间（单位：毫秒）内，判断主机与该IP对应的主机之间是否连通。

2.UDP通信

2.1 `DatagramSocket` 类：创建客户端和服务端

P1 构造器

构造器	说明
<code>public DatagramSocket()</code>	创建 客户端 的 <code>Socket</code> 对象，系统会为其随机分配一个端口号。
<code>public DatagramSocket(int port)</code>	创建 服务端 的 <code>Socket</code> 对象，并指定端口号。

P2 常用方法

方法	说明
<code>void send(DatagramPacket dp)</code>	发送数据包。
<code>void receive(DatagramPacket dp)</code>	使用数据包接收数据。

2.2 `DatagramPacket`：创建数据包

P1 构造器

构造器	说明
<code>public DatagramPacket(byte[] buf, int length, InetAddress, int port)</code>	创建用于发送数据的数据包对象，必须指定服务端IP和端口号。
<code>public Datagram(byte[] buf, int length)</code>	创建用于接收数据的数据包对象。

P2 常用方法

方法	说明
<code>int getLength()</code>	获取当前数据包实际接收到的字节数。
<code>InetAddress getAddress()</code>	获取发送当前数据包的客户端的IP地址对象。
<code>int getPort()</code>	获取发送当前数据包的客户端端口。

2.3 一发一收

⚠ Caution

用完 `Socket` 后记得关闭资源。

- `Server`:

```
1 package udp;
2
3 import java.io.IOException;
4 import java.net.DatagramPacket;
5 import java.net.DatagramSocket;
6 import java.net.InetAddress;
7 import java.net.UnknownHostException;
8
9 public class Server {
10     public static final InetAddress IP;
11     public static final int PORT = 6666;
12     public static final int LENGTH = 1024 * 64; // 64KB
13
14     static {
15         try {
16             IP = InetAddress.getLocalHost();
17         } catch (UnknownHostException e) {
18             throw new RuntimeException(e);
19         }
20     }
21 }
```

```

20     }
21
22     public static void main(String[] args) {
23         System.out.println("Server starts");
24         try (
25             DatagramSocket socket = new DatagramSocket(PORT); //
用完自动关闭资源
26         ) {
27             byte[] buf = new byte[LENGTH];
28             DatagramPacket packet = new DatagramPacket(buf,
LENGTH);
29
30             socket.receive(packet);
31             System.out.println("Server receives a packet from
Client, the contents are as follows:");
32             System.out.println("Client IP: " +
packet.getAddress().getHostAddress());
33             System.out.println("Client Port: " + packet.getPort());
34             int length = packet.getLength();
35             System.out.println(new String(buf, 0, length)); // 接收多
少，就倒出多少
36         } catch (IOException e) {
37             throw new RuntimeException(e);
38         }
39     }
40 }

```

- **Client:**

```

1  package udp;
2
3  import java.io.IOException;
4  import java.net.*;
5
6  public class Client {
7      public static void main(String[] args) {
8
9          try (
10             DatagramSocket socket = new DatagramSocket(); // 用
完后自动关闭资源
11         ) {
12             byte[] buf = "Hello Server, I'm Client".getBytes();
13             DatagramPacket packet = new DatagramPacket(
14                 buf, buf.length, Server.IP, Server.PORT);

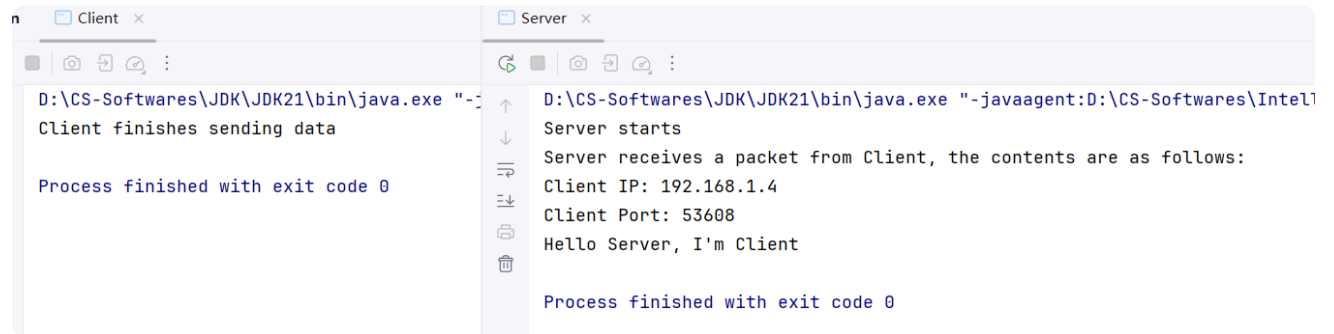
```

```

15
16         socket.send(packet);
17         System.out.println("Client finishes sending data");
18     } catch (IOException e) {
19         throw new RuntimeException();
20     }
21 }
22 }

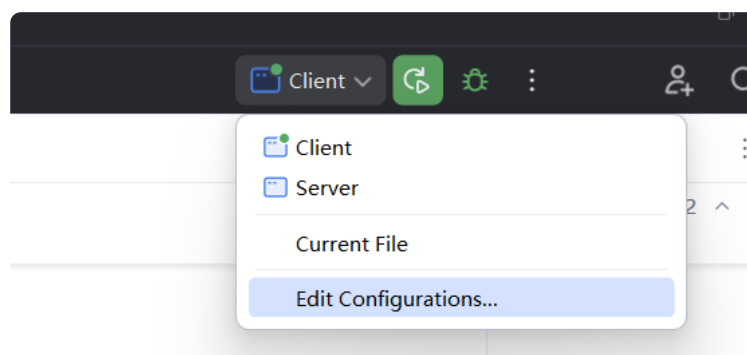
```

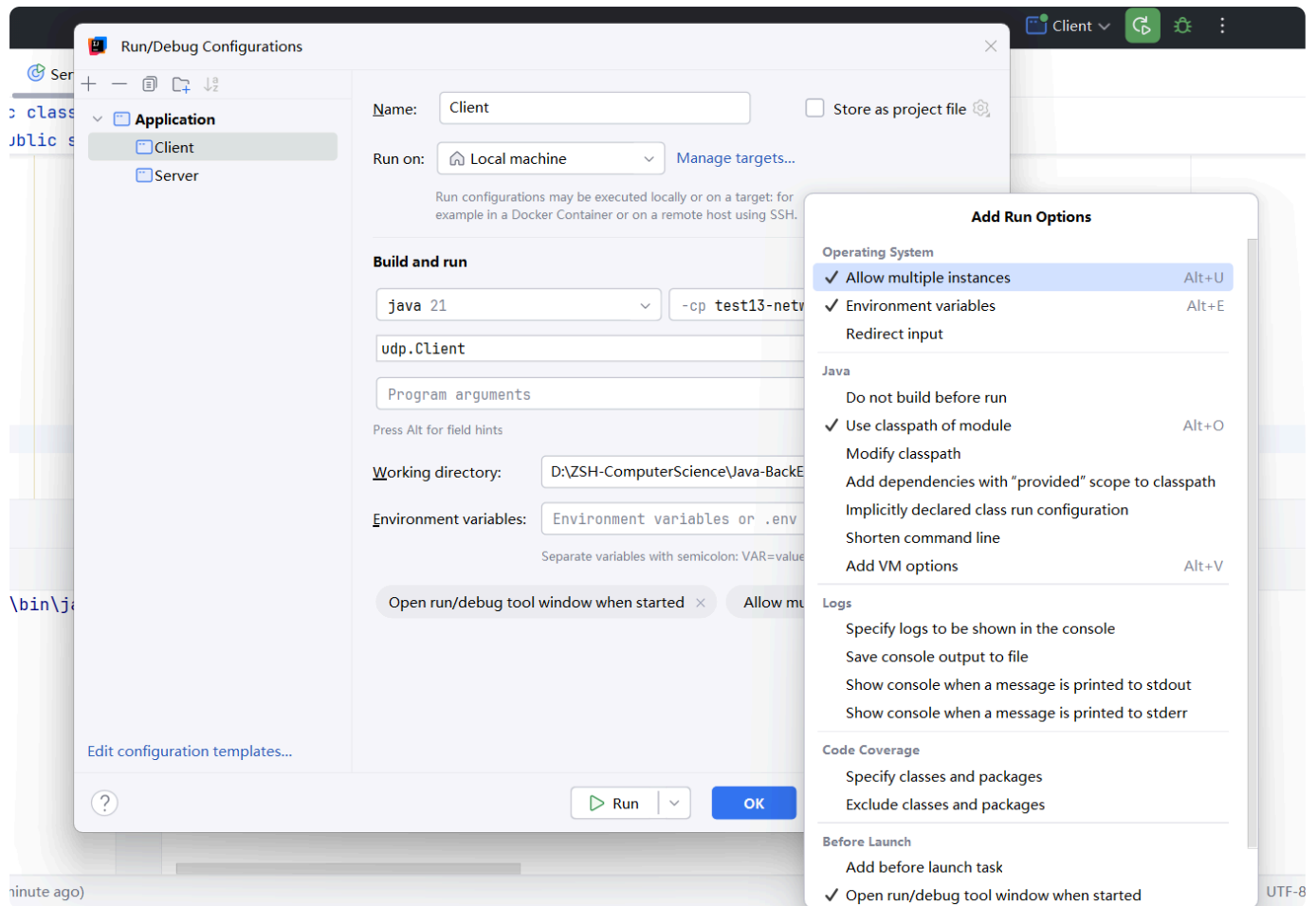
- 先启动服务端，再启动客户端，控制台输出如下：

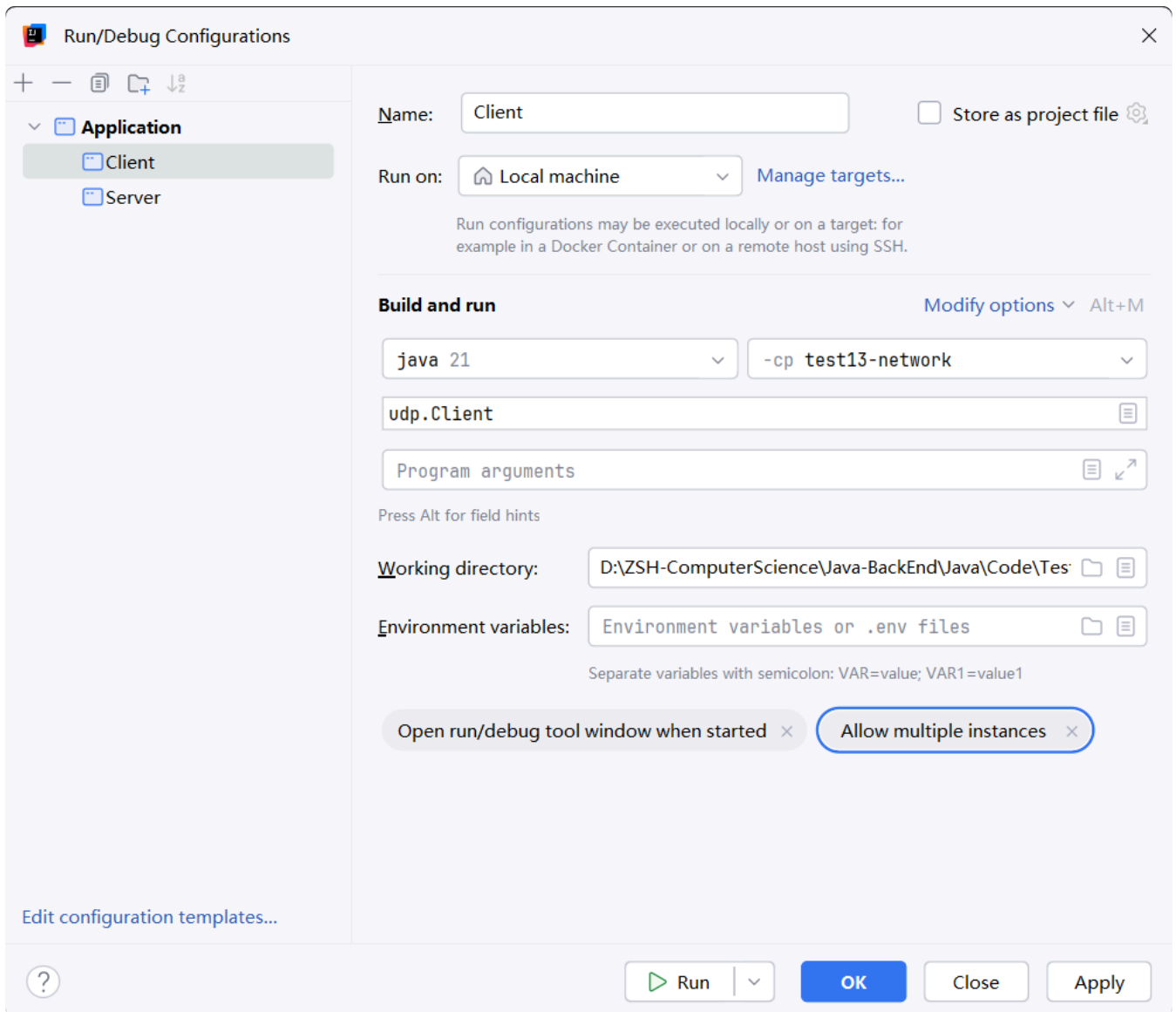


2.4 多发多收

P1 IDEA中允许客户端多开的设置步骤







最后点击Apply或OK即可。

P2 代码实现

- `Server` :

```
1 package udp;
2
3 import java.io.IOException;
4 import java.net.DatagramPacket;
5 import java.net.DatagramSocket;
6 import java.net.InetAddress;
7 import java.net.UnknownHostException;
8
9 public class Server {
```

```

10     public static final InetAddress IP;
11     public static final int PORT = 6666;
12     public static final int LENGTH = 1024 * 64; // 64KB
13
14     static {
15         try {
16             IP = InetAddress.getLocalHost();
17         } catch (UnknownHostException e) {
18             throw new RuntimeException(e);
19         }
20     }
21
22     public static void main(String[] args) {
23         System.out.println("Server starts");
24         try (
25             DatagramSocket socket = new DatagramSocket(PORT);
26         ) {
27             byte[] buf = new byte[LENGTH];
28             DatagramPacket packet = new DatagramPacket(buf,
29                 LENGTH);
30
31             int count = 1;
32             while (true) {
33                 socket.receive(packet);
34                 System.out.println("====Packet " + (count++) +
35                     "====");
36                 System.out.println("Client IP: " +
37                     packet.getAddress().getHostAddress());
38                 System.out.println("Client Port: " +
39                     packet.getPort());
40                 int length = packet.getLength();
41                 System.out.println("Contents: " + new String(buf,
42                     0, length)); // 接收多少，就倒出多少
43                 System.out.println("-----");
44             }
45         } catch (IOException e) {
46             throw new RuntimeException(e);
47         }
48     }

```

- Client:


```

1 package udp;
2
3 import java.io.IOException;
4 import java.net.*;
5 import java.util.Scanner;
6
7 public class Client {
8     public static void main(String[] args) {
9
10         try (
11             DatagramSocket socket = new DatagramSocket();
12         ) {
13             Scanner sc = new Scanner(System.in);
14             while (true) {
15                 System.out.println("please input a message:");
16                 String msg = sc.nextLine();
17                 if (msg.equals("exit")) {
18                     System.out.println("Client exits
19 successfully");
20                     break;
21                 }
22                 byte[] buf = msg.getBytes();
23                 DatagramPacket packet = new DatagramPacket(
24                     buf, buf.length, Server.IP, Server.PORT);
25                 socket.send(packet);
26             }
27         } catch (IOException e) {
28             throw new RuntimeException();
29         }
30     }
31 }

```

- 先启动服务端，再分别启动3个客户端，控制台输出如下：

Run Client x

D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwares\1
please input a message:
This is Client 1
please input a message:
exit
Client exits successfully

Process finished with exit code 0

Client x

D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwares\1
please input a message:
This is Client 2
please input a message:

Client x

D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwares\1
please input a message:
This is Client 3
please input a message:

Server x

D:\CS-Softwares\JDK\JDK21\bin\java.e
Server starts
====Packet 1====
Client IP: 192.168.1.4
Client Port: 65024
Contents: This is Client 1

====Packet 2====
Client IP: 192.168.1.4
Client Port: 55234
Contents: This is Client 2

====Packet 3====
Client IP: 192.168.1.4
Client Port: 65025
Contents: This is Client 3

3A.单客户端TCP通信

3A.1 原理图



3A.2 `Socket` 类：开发客户端

P1 构造器

构造器	说明
<code>public Socket(String host, int port)</code>	根据服务端IP和端口号，请求与服务端建立连接，连接成功后返回客户端 <code>Socket</code> 对象。

P2 常用方法

方法	说明
<code>OutputStream getOutputStream()</code>	获取字节输出流对象。
<code>InputStream getInputStream()</code>	获取字节输入流对象。

3A.3 `ServerSocket` 类：开发服务端

P1 构造器

构造器	说明
<code>public ServerSocket(int port)</code>	为服务端程序注册端口。

P2 常用方法

方法	说明
<code>Socket accept()</code>	阻塞等待客户端的连接请求，一旦与某个客户端连接成功，就返回服务端 <code>Socket</code> 对象。

3A.4 代码演示（一发一收）

- **Server:**

```
1 package tcp;
2
3 import java.io.DataInputStream;
4 import java.io.IOException;
5 import java.net.InetAddress;
6 import java.net.ServerSocket;
7 import java.net.Socket;
8 import java.net.UnknownHostException;
9
10 public class Server {
11     public static final String IP;
12     public static final int PORT = 6666;
13
14     static {
15         try {
16             IP = InetAddress.getLocalHost().getHostAddress();
17         } catch (UnknownHostException e) {
18             throw new RuntimeException(e);
19         }
20     }
21
22     public static void main(String[] args) {
23         try (
24             ServerSocket serverSocket = new ServerSocket(PORT);
25             Socket socket = serverSocket.accept();
26             DataInputStream dis = new
DataInputStream(socket.getInputStream());
27         ) {
28             System.out.println("Client Address: " +
socket.getRemoteSocketAddress());
29             System.out.println("Contents: " + dis.readUTF());
30
31         } catch (IOException e) {
32             throw new RuntimeException(e);
33         }
34     }
35 }
```

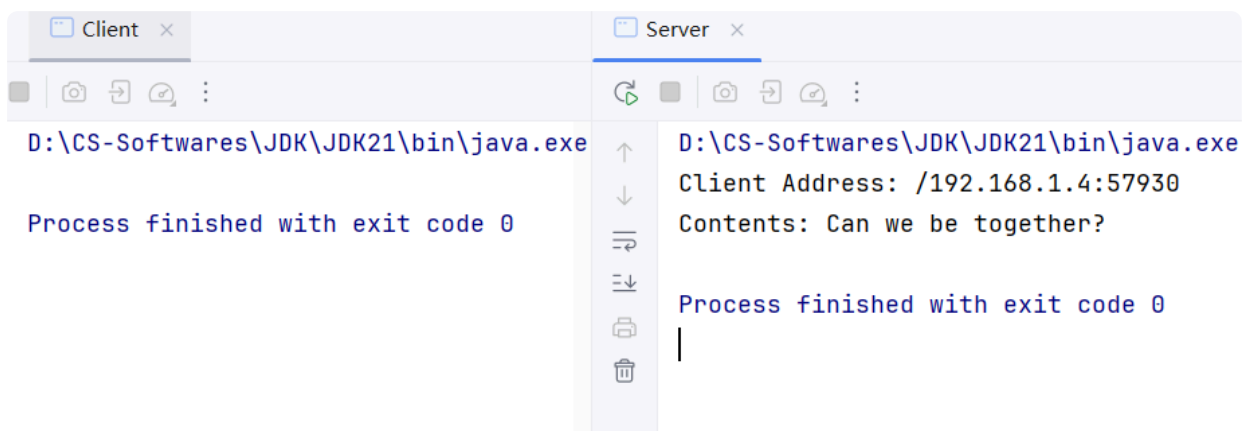
- **Client:**

```

1 package tcp;
2
3 import java.io.DataOutputStream;
4 import java.io.IOException;
5 import java.net.Socket;
6 import java.net.UnknownHostException;
7
8 public class Client {
9     public static void main(String[] args) {
10         try (
11             Socket socket = new Socket(Server.IP, Server.PORT);
12             DataOutputStream dos = new
DataOutputStream(socket.getOutputStream()); // 把低级流包装成高级流
13         ) {
14
15             dos.writeUTF("Can we be together?");
16
17         } catch (UnknownHostException e) {
18             throw new RuntimeException(e);
19         } catch (IOException e) {
20             throw new RuntimeException(e);
21         }
22     }
23 }

```

- 先启动服务端，再启动客户端，控制台输出如下：



3A.5 单客户端多发多收注意事项

1. 客户端每写一次数据，就要刷新一下。

```
Client.java x
26
27 // 一旦用户输入了exit, 就退出客户端程序
28 if("exit".equals(msg)){
29     System.out.println("欢迎您下次光临! 退出成功! ");
30     dos.close();
31     socket.close();
32     break;
33 }
34
35 // 4、开始写数据出去了
36 dos.writeUTF(msg);
37 dos.flush();
38 }
39 }
40 }
```

2.若客户端突然离线, 则服务端必须处理异常, 关闭相关资源。

```
Server.java x
15 // 1、创建ServerSocket的对象, 同时为服务端注册端口。
16 ServerSocket serverSocket = new ServerSocket( port: 8888);
17
18 // 2、使用serverSocket对象, 调用一个accept方法, 等待客户端的连接请求
19 Socket socket = serverSocket.accept();
20
21 // 3、从socket通信管道中得到一个字节输入流。
22 InputStream is = socket.getInputStream();
23
24 // 4、把原始的字节输入流包装成数据输入流
25 DataInputStream dis = new DataInputStream(is);
26
27 while (true) {
28     try {
29         // 5、使用数据输入流读取客户端发送过来的消息
30         String rs = dis.readUTF();
31         System.out.println(rs);
32     } catch (Exception e) {
33         System.out.println(socket.getRemoteSocketAddress() + "离线了! ")
34         dis.close();
35         socket.close();
36         break;
37     }
38 }
```

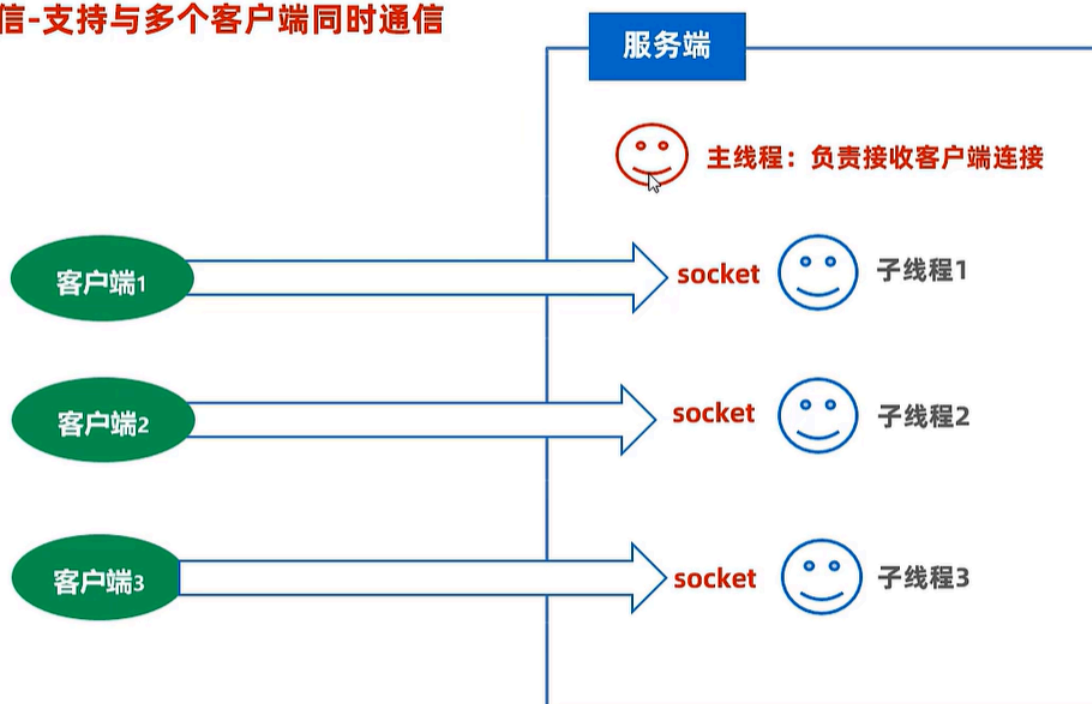
3B.多客户端TCP通信

3B.1 3A代码的局限性

3A代码无法支持多客户端与服务端的TCP通信，根本原因是服务端目前只有一个主线程，只能处理一个客户端的信息。

3B.2 重构思路：引入多线程

TCP通信-支持与多个客户端同时通信



3B.3 代码实现

- `ServerReaderThread` :

```
1 package tcp;
2
3 import java.io.DataInputStream;
4 import java.io.IOException;
5 import java.net.Socket;
6
7 public class ServerReaderThread extends Thread {
```

```

8     private Socket socket;
9
10    public ServerReaderThread(Socket socket) {
11        this.socket = socket;
12    }
13
14    @Override
15    public void run() {
16        try (
17            DataInputStream dis = new
DataInputStream(socket.getInputStream());
18        ) {
19            while (true) {
20                try {
21                    System.out.println(
22                        "Client Address: " +
socket.getRemoteSocketAddress() +
23                        "\nContents: " + dis.readUTF()
+ "\n");
24                } catch (IOException e) {
25
26                    System.out.println(socket.getRemoteSocketAddress() + "
terminals");
27
28                    dis.close();
29                    socket.close();
30                    break;
31                }
32            }
33        } catch (IOException e) {
34            throw new RuntimeException(e);
35        }
36    }

```

- **Server:**

```

1  package tcp;
2
3  import java.io.IOException;
4  import java.net.InetAddress;
5  import java.net.ServerSocket;
6  import java.net.Socket;
7  import java.net.UnknownHostException;
8

```



```

9  public class Server {
10     public static final String IP;
11     public static final int PORT = 6666;
12
13     static {
14         try {
15             IP = InetAddress.getLocalHost().getHostAddress();
16         } catch (UnknownHostException e) {
17             throw new RuntimeException(e);
18         }
19     }
20
21     public static void main(String[] args) throws IOException {
22         System.out.println("Server starts.");
23         ServerSocket serverSocket = new ServerSocket(PORT);
24
25         while (true) {
26             Socket socket = serverSocket.accept();
27             new ServerReaderThread(socket).start();
28         }
29     }
30 }

```

- Client:

```

1  package tcp;
2
3  import java.io.DataOutputStream;
4  import java.io.IOException;
5  import java.net.Socket;
6  import java.net.UnknownHostException;
7  import java.util.Scanner;
8
9  public class Client {
10     public static void main(String[] args) {
11         Scanner sc = new Scanner(System.in);
12         try (
13             Socket socket = new Socket(Server.IP, Server.PORT);
14             DataOutputStream dos = new
DataOutputStream(socket.getOutputStream()); // 把低级流包装成高级流
15         ) {
16             while (true) {
17                 System.out.println("Please input a message:");
18                 String msg = sc.nextLine();

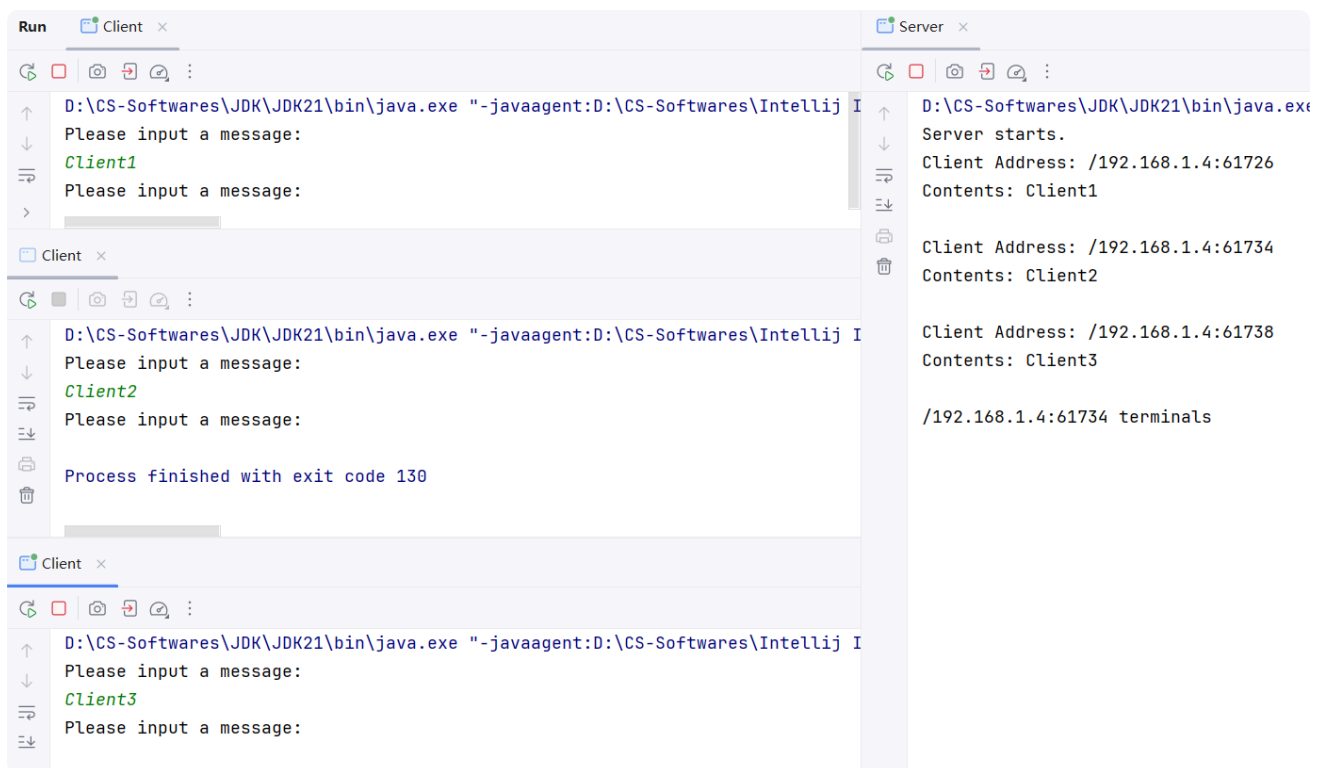
```

```

19         if (msg.equals("exit")) {
20             System.out.println("Client exits
successfully!");
21             break;
22         }
23         dos.writeUTF(msg);
24         dos.flush();
25     }
26 } catch (IOException e) {
27     throw new RuntimeException(e);
28 }
29 }
30 }

```

- 先启动服务端，再分别启动3个客户端，控制台输出如下：

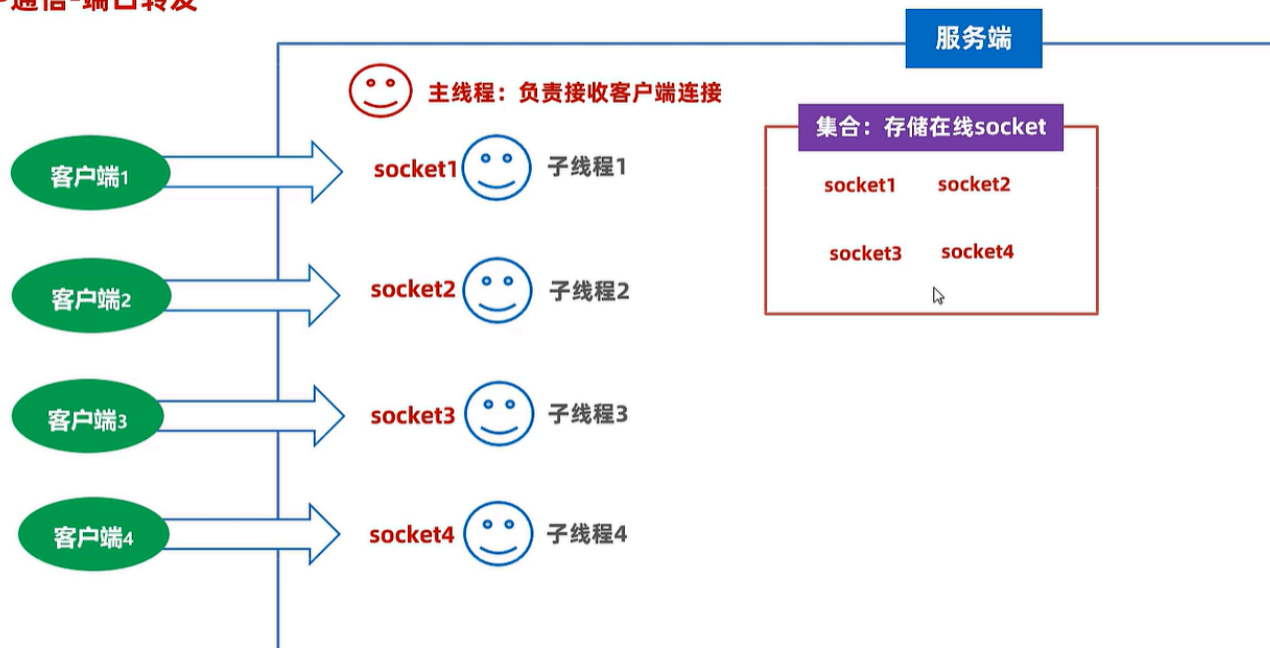


4.TCP通信综合案例

4.1 即时通信群聊

P1 原理图

TCP通信-端口转发



P2 代码实现

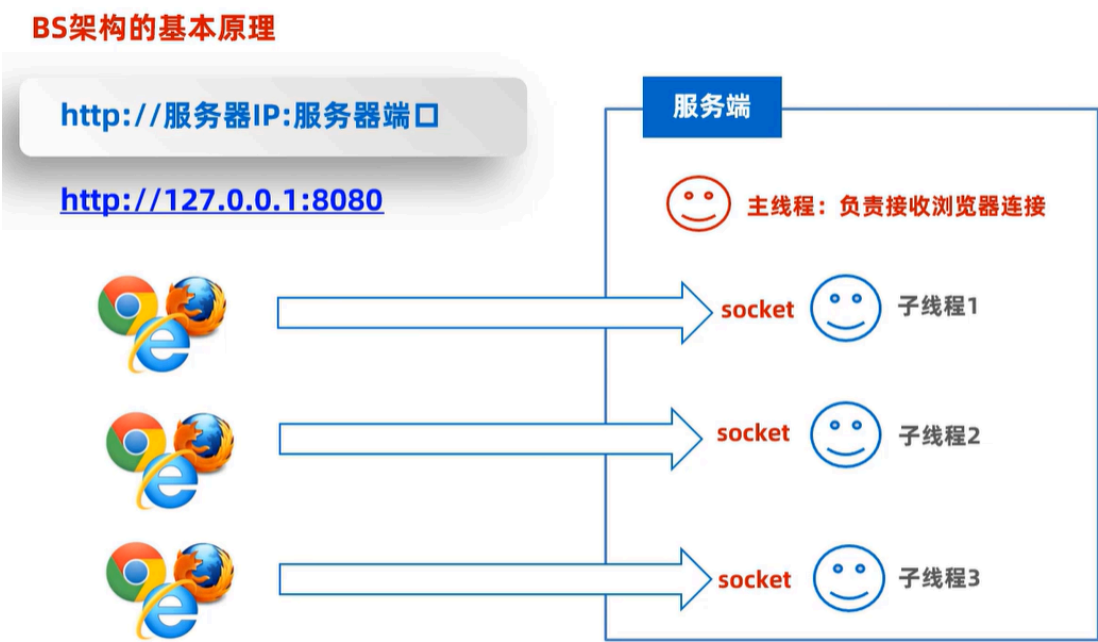
具体代码参见项目: [wechat](#)

4.2 简易版BS架构

P1 需求

从浏览器中访问服务器，并立即让服务器响应一个简单网页渲染到浏览器上，网页内容是“黑马程序员666”。

P2 原理图



注意：服务器必须给浏览器响应HTTP协议规定的格式，否则浏览器不识别返回的数据。

- HTTP协议规定：响应给浏览器的数据格式必须满足如下格式

协议版本	空格	状态码	空格	状态符	回车换行
头部字段名	:	值	回车换行		
.....					
头部字段名	:	值	回车换行		
回车换行					
响应正文（真正给浏览器展示的网页数据）					

HTTP/1.1 200 OK
Content-Type : text/html;charset=UTF-8
.....
.....
必须换行
 黑马程序员磊哥

P3 代码实现

具体代码参见项目：[simpleBS](#)

Chrome浏览器访问localhost:8888，网页展示如下：

黑马程序员666

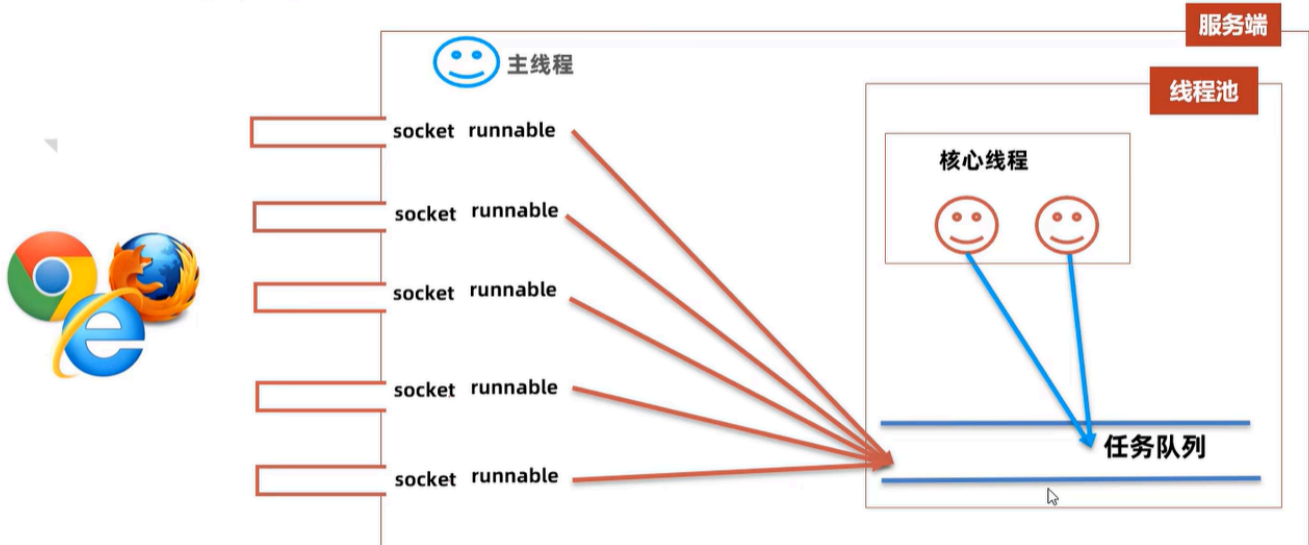
5.拓展

Q：每次网络请求都开一个新线程，到底好不好？

A：高并发时，容易宕机！

解决方案：使用线程池进行优化。

使用线程池进行优化



```
Server.java x ServerReaderRunnable.java x Thread.java x
9
10 /**
11  * 目标：完成TCP通信快速入门-服务端开发：要求实现与多个客户端同时通信。
12  */
13 public class Server {
14     public static void main(String[] args) throws Exception {
15         System.out.println("-----服务端启动成功-----");
16         // 1、创建ServerSocket的对象，同时为服务端注册端口。
17         ServerSocket serverSocket = new ServerSocket(port: 8080);
18
19         // 创建一个线程池，负责处理通信管道的任务。
20         ThreadPoolExecutor pool = new ThreadPoolExecutor(corePoolSize: 16,
21             new ArrayBlockingQueue<>(capacity: 8), Executors.defaultThreadFactory(),
22             new ThreadPoolExecutor.AbortPolicy());
23
24         while (true) {
25             // 2、使用serverSocket对象，调用一个accept方法，等待客户端的连接请求。
26             Socket socket = serverSocket.accept();
27
28             // 3、把这个客户端对应的socket通信管道，交给一个独立的线程负责处理。
29             pool.execute(new ServerReaderRunnable(socket));
30         }
31     }
32 }
33
34
35
36 package com.itheima.d9_tcp6;
37
38 import ...
39
40 public class ServerReaderRunnable implements Runnable{
41     private Socket socket;
42     public ServerReaderRunnable(Socket socket) { this.socket = socket; }
43     @Override
44     public void run() {
45         // 立即响应一个网页内容：“黑马程序员”给浏览器展示。
46         try {
47             OutputStream os = socket.getOutputStream();
48             PrintStream ps = new PrintStream(os);
49             ps.println("HTTP/1.1 200 OK");
50             ps.println("Content-Type:text/html;charset=UTF-8");
51             ps.println(); // 必须换行
52             ps.println("<div style='color:red;font-size:24px;'>黑马程序员");
53             ps.close();
54             socket.close();
55         } catch (Exception e) {
56             e.printStackTrace();
57         }
58     }
59 }
```